LLD Practice Platform (MVP)

A focused, pragmatic Low-Level Design (LLD) practice platform built for software engineers preparing for object-oriented design interviews.

Unlike algorithmic platforms (LeetCode) or generic AI chat interfaces, this platform focuses specifically on the **iterative learner journey**:

```
Choose Problem ➔ Read Requirements & Boundaries ➔ Draft Design ➔ Submit ➔ Get Grounded Feedback ➔ R
```

---

1. Quick Start

Prerequisites

- **Language**: Python 3.10+
- **Dependencies**: **Zero external pip dependencies**. Uses only the standard library (`ast` , `concurrent.futures` , `dataclasses` , `http.server` , `unittest`).

Running the Full Test Suite

Run all 15 unit and integration tests:

```
python -m unittest discover -s tests -v
```

Running the End-to-End Automated Demo

Walks through an end-to-end learner journey (Problem Selection ➔ Naive Attempt 1 with God Class ➔ Detailed Feedback ➔ Refactored Attempt 2 ➔ Attempt Progression Diff):

```
python -m lld_platform.cli.runner --demo
```

Running the Interactive CLI

Launch the terminal interactive interface:

```
python -m lld_platform.cli.runner
```

Running the Web Dashboard (Optional)

Launch the lightweight built-in HTTP server and open http://localhost:8080 in your browser:

```
python -m lld_platform.web.server
```

---

2. Supported Problems in MVP Catalog

The platform includes 4 seed problems with clear requirements and explicit scope boundaries:

1. **Design a Multi-Floor Parking Lot** (`parking-lot`)
   - *Entities*: `Vehicle` , `ParkingSpot` , `ParkingLot` , `ParkingTicket` , `PricingStrategy`
   - *Key Focus*: Spot assignment, ticket lifecycle, and Strategy pattern for fee calculation.
2. **Design an Elevator Control System** (`elevator-system`)
   - *Entities*: `ElevatorCar` , `ElevatorController` , `Floor` , `Request` , `ElevatorState`
   - *Key Focus*: State modeling, separation of car vs controller, dispatching algorithm.
3. **Design Splitwise (Expense Sharing)** (`splitwise-expenses`)
   - *Entities*: `User` , `Expense` , `SplitStrategy` , `Group`
   - *Key Focus*: Strategy pattern for split calculations, bilateral balance tracking.
4. **Design an In-Memory Cache with Eviction** (`in-memory-cache`)
   - *Entities*: `Cache` , `EvictionPolicy` , `CacheEntry`
   - *Key Focus*: Decoupling cache storage from pluggable eviction policy.

---

3. Key Design Decisions

1. **Submission Format: Python AST-Parseable Code**
   - *Justification*: In LLD interviews, candidates write code. Parsing submissions via Python's built-in `ast` module enables 100% deterministic, zero-dependency static inspection (class detection, method counting, inheritance checking, and line number citations) without relying on brittle regex.
2. **Two-Tier Evaluation Engine (Deterministic + Qualitative)**
   - *Deterministic*: Checks baseline structural rules (required entities, required operations, Single Responsibility / God class threshold).
   - *Qualitative*: Pluggable `LLMReviewer` interface evaluates design pattern choices (Strategy, Factory, State), extensibility, and concurrency risks.
3. **Attempt History & Progression Diffing**
   - Tracks attempts per learner and problem.
   - Computes a progression summary between Attempt $N-1$ and Attempt N, surfacing resolved issues, persisting warnings, and score deltas.
4. **Open/Closed Principle Extensibility**
   - New deterministic checks implement `EvaluationRule` and plug directly into `DeterministicEvaluator.add_rule()` without modifying existing code.
   - LLM reviewer is behind an interface (`LLMReviewer`), stubbed by default for fast, deterministic testing.
5. **Defensive Failure Handling**
   - AI timeouts and network glitches are isolated using `concurrent.futures.ThreadPoolExecutor` : deterministic feedback is preserved even if the LLM reviewer times out.
   - Severe unhandled exceptions transition attempt to `FAILED` status, logging diagnostic details and allowing retry.

4. Project Structure

```
lld-practice-platform/
├── README.md
├── AI_USAGE.md
├── docs/
│   ├── research_note.md         # Learner problem & product direction
│   └── design_note.md           # Architecture, domain model, trade-offs
├── lld_platform/
│   ├── domain/
│   │   ├── models.py            # Problem, Submission, Attempt, AttemptStatus
│   │   └── feedback.py          # FeedbackReport, CheckFinding, ProgressionSummary
│   ├── problems/
│   │   └── catalog.py           # Seed problem definitions & evaluation specs
│   ├── evaluation/
│   │   ├── base.py              # EvaluationRule, LLMReviewer, Evaluator interfaces
│   │   ├── rules.py             # AST rules: GodClass, RequiredEntities, Operations
│   │   ├── deterministic.py     # DeterministicEvaluator coordinating rules
│   │   ├── llm_reviewer.py      # StubbedLLMReviewer & MockFailingLLMReviewer
│   │   └── pipeline.py          # EvaluationPipeline with timeout isolation
│   ├── repository/
│   │   ├── base.py              # AttemptRepository, ProblemRepository interfaces
│   │   └── memory.py            # Thread-safe in-memory implementations
│   ├── service/
│   │   └── platform_service.py  # Orchestrates attempts, evaluation, and progression
│   ├── cli/
│   │   └── runner.py            # Interactive CLI & automated demo
│   └── web/
│       └── server.py            # Lightweight zero-dependency HTTP server & UI
├── samples/
│   ├── parking_lot_v1_bad.py    # Naive parking lot (God class, missing entities)
│   ├── parking_lot_v2_good.py   # Refactored parking lot (Clean classes, Strategy)
│   └── elevator_submission.py   # Clean elevator system implementation
└── tests/
    ├── test_domain.py
    ├── test_deterministic_evaluator.py
    ├── test_evaluation_pipeline.py
    ├── test_failure_handling.py
    └── test_extensibility.py
```

5. MVP Limitations

- **Language Scope**: Currently evaluates Python code submissions. Java/C++ support would require dedicated AST parsers or LSP integration.
- **In-Memory Persistence**: Storage is currently in-memory. Server restarts reset attempt history unless dumped to disk.
- **Synchronous Web Worker**: The web server handles submissions synchronously within the request lifecycle; in a multi-tenant production environment, evaluations should be enqueued into an async task queue.

6. What I Would Do With Another Day

1. **Live LLM Integration**: Connect `OpenAICompatibleReviewer` with an active Gemini/OpenAI API key and test dynamic feedback variations.
2. **Interactive Visual Class Diagram Generation**: Automatically generate an interactive visual UML diagram from the learner's submitted AST using an embedded Mermaid or SVG renderer.
3. **AST Linting & Static Typing**: Add a pluggable rule checking for PEP 484 type annotations on method parameters and return types.
4. **Persistent SQLite Store**: Swap `InMemoryAttemptRepository` with an SQLite-backed repository to persist attempt history and code diffs across restarts.
5. **Multi-File Submissions**: Support multi-file package designs where candidates separate classes into separate modules (e.g. `parking/models.py`, `parking/strategies.py`).

AI Usage & Collaborative Architectural Decisions

This document outlines 4 meaningful AI-assisted decisions made during the design and implementation of the LLD Practice Platform MVP, detailing what was proposed, what was accepted or rejected, and the underlying engineering rationale.

Decision 1: Submission Format (Mermaid Class Diagram DSL vs. AST-Parseable Python Code)

- **What the AI Suggested**: Use a Mermaid class diagram text format (e.g. `classDiagram \n class ParkingLot { +parkVehicle() }`) or a structured YAML schema, arguing that diagrams match whiteboard design rounds and are easy to validate with regex.
- **Accepted or Rejected**: **Rejected.** Selected Python class outlines parsed via Python's built-in `ast` module.
- **Engineering Rationale**: In senior LLD and machine coding interviews, candidates write code: classes, attributes, methods, type annotations, and design patterns. Testing candidate proficiency in a diagramming DSL (like Mermaid or PlantUML) introduces unnecessary friction and tests syntax memorization rather than object-oriented design. Furthermore, regex-based validation is notoriously brittle. Python's built-in Abstract Syntax Tree (`ast`) provides zero-dependency, 100% deterministic static analysis: accurate method counting, attribute tracking, inheritance trees, and exact source line numbers for citations.

Decision 2: Asynchronous Job Architecture (Celery/Redis vs. In-Process ThreadPoolExecutor)

- **What the AI Suggested**: Build a full asynchronous job queue using Redis and Celery to manage attempt evaluations, arguing this is how production platforms manage slow AI operations.
- **Accepted or Rejected**: **Rejected.** Implemented an in-process `concurrent.futures.ThreadPoolExecutor` with timeout bounds inside `EvaluationPipeline`.
- **Engineering Rationale**: The assignment constraints explicitly stated: "*Monolith only. No microservices... Do not build a distributed job system.*" Introducing Redis and worker daemons creates heavy installation overhead, external network requirements, and operational complexity for a 2-day take-home MVP. Python's standard `concurrent.futures` isolates slow/hanging LLM calls behind a strict timeout without requiring any external infrastructure.

Decision 3: Evaluation Engine (Single-Prompt LLM vs. Split Deterministic AST + Qualitative Review)

- **What the AI Suggested**: Send the candidate's code and problem statement to a single LLM prompt, asking the model to return a combined JSON response containing scores, detected classes, God class warnings, and advice.
- **Accepted or Rejected**: **Rejected the single-prompt approach; Accepted the two-tier split architecture.**
- **Engineering Rationale**: LLMs are non-deterministic, hallucinate entity presence, and are notoriously unreliable at counting methods or catching subtle omissions. In an interview assessment tool, grading must be explainable and consistent. By anchoring non-negotiables (required entities, required operations, God class thresholds, line citations) in deterministic AST rules, feedback is repeatable and trustworthy. The LLM is used strictly where it excels: qualitative critique of design patterns, trade-offs, and extensibility.

Decision 4: Failure Isolation & Fallback Degradation

- **What the AI Suggested**: If the LLM reviewer experiences an error or times out, fail the entire attempt and mark its status as `FAILED`.

- **Accepted or Rejected**: **Refined into a tiered fallback strategy.**
- **Engineering Rationale**: If an external LLM times out or is unreachable, the learner should not lose their structural feedback. In our implementation, `EvaluationPipeline` traps LLM timeouts and network errors gracefully: the deterministic AST checks are preserved and displayed, while the qualitative section displays a clear notice that AI advice timed out. Only catastrophic engine failures (e.g. out-of-memory or unhandled worker exceptions) mark the attempt as `FAILED`, and the platform explicitly provides `retry_attempt(id)` to allow retrying without re-entering code.